

RAPPORT DE PROJET — sujet 3

Customer 360

Graphe de Relations & Entrepôt de Données

Neo4j · PostgreSQL · dbt · Power BI · Python

Module : Système Business Intelligence

Encadrant : Pr. HRIMECH HAMID

Année académique : 2025/2026

Membres du groupe :

- AMARA Yassine
- SMAOUI Mohamed

Table des matières

1	Introduction Générale	3
1.1	Contexte du Projet	3
1.2	Problématique	3
1.3	Objectifs	3
1.4	Organisation du Rapport	3
2	Architecture Globale du Système	5
2.1	Vue d'Ensemble	5
2.2	Technologies Utilisées	6
2.3	Pipeline de Données	6
3	Modélisation du Graphe Neo4j	7
3.1	Présentation de Neo4j	7
3.2	Modèle de Graphe	7
3.2.1	Nœuds (Nodes)	7
3.2.2	Relations	8
3.3	Import des Données dans Neo4j	8
3.3.1	Création des nœuds Client	8
3.3.2	Création des Relations Transactions	8
3.4	Visualisation du Graphe	9
4	Entrepôt de Données PostgreSQL	10
4.1	Présentation de l'Architecture en Étoile	10
4.2	Schéma en Étoile — Customer 360	10
4.3	Tables de l'Entrepôt	10
4.3.1	Table de Faits : fct_transactions	10
4.3.2	Dimensions	11
4.4	Vue Matérialisée — Lien Graphe et Entrepôt	11
5	Transformations dbt	13
5.1	Présentation de dbt	13
5.2	Structure du Projet dbt	13
5.3	Couche Staging : stg_banking_data	13
5.4	Exécution du Pipeline dbt	14
5.5	Graphe de Dépendances dbt (DAG)	14
6	Détection de Communautés — Algorithme Louvain	16
6.1	Principe de l'Algorithme Louvain	16
6.2	Implémentation avec NetworkX et Python	16

6.3	Résultats de la Détection de Communautés	18
7	Dashboard Power BI	19
7.1	Connexion à PostgreSQL	19
7.2	Modèle de Données dans Power BI	19
7.3	KPIs Implémentés	20
7.4	Visualisations du Dashboard	20
8	Tests et Performances	22
8.1	Tests de Qualité des Données	22
8.2	Performances du Pipeline	22
9	Conclusion	23
9.1	Bilan du Projet	23

1. Introduction Générale

1.1 Contexte du Projet

Dans un environnement bancaire de plus en plus compétitif, la maîtrise de la donnée client constitue un avantage stratégique majeur. Les banques modernes collectent des volumes considérables de données provenant de sources hétérogènes : transactions financières, interactions avec les centres d'appel, navigations sur les plateformes web et mobiles, souscriptions de produits financiers, et retours clients.

Le concept de **Customer 360** désigne une vision unifiée et exhaustive du client, consolidant l'ensemble de ces sources en une représentation cohérente permettant d'analyser les comportements, d'anticiper les besoins et de personnaliser les offres.

1.2 Problématique

Comment une banque peut-elle fusionner des données clients hétérogènes (transactions, interactions multicanal) pour construire une plateforme analytique unifiée capable de :

- Modéliser les relations complexes entre clients, comptes, agences et produits,
- Calculer des KPIs métier (solde moyen, turnover transactionnel),
- Détecter des communautés clients pour la personnalisation,
- Recommander des produits financiers adaptés.

1.3 Objectifs

1. Modéliser un **graphe de relations** (client, compte, agence, produit) et l'importer dans Neo4j
2. Construire un **entrepôt de données en étoile** dans PostgreSQL via dbt
3. **Lier** le graphe et l'entrepôt via des vues matérialisées
4. Appliquer l'algorithme de **détection de communautés Louvain** (NetworkX)
5. Développer un **dashboard interactif** Power BI filtré par communautés
6. Implémenter un système de **recommandation de produits**

1.4 Organisation du Rapport

Ce rapport est structuré comme suit : le Chapitre 2 présente l'architecture globale du système, le Chapitre 3 détaille la modélisation du graphe Neo4j, le Chapitre 4 décrit l'entrepôt de données PostgreSQL, le Chapitre 5 couvre les transformations dbt, le Chapitre

6 explique la détection de communautés, le Chapitre 7 présente le dashboard Power BI, et le Chapitre 8 décrit le système de recommandation.

2. Architecture Globale du Système

2.1 Vue d'Ensemble

L'architecture du projet Customer 360 suit un pipeline de données en plusieurs couches, depuis l'ingestion des données brutes jusqu'à la visualisation des KPIs dans Power BI.

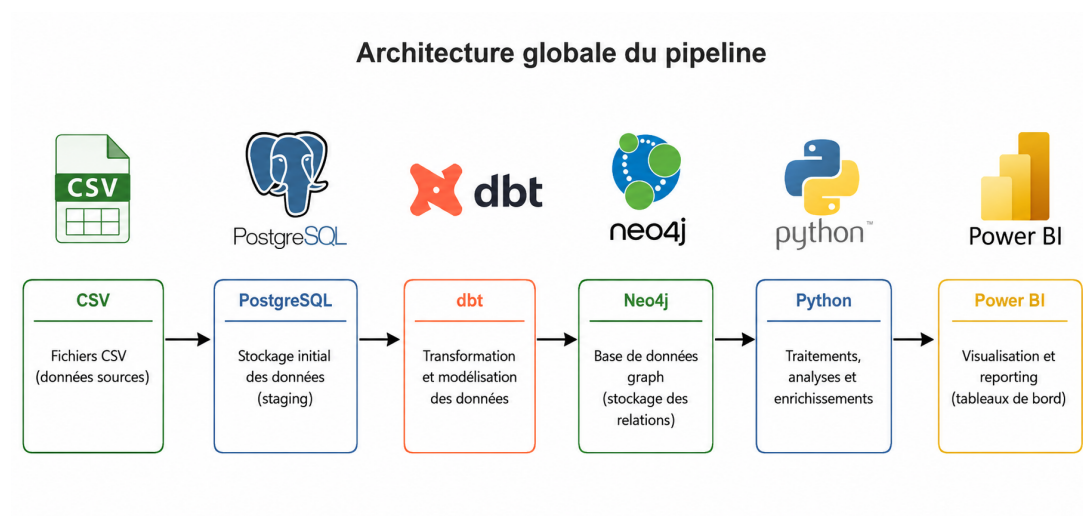


FIGURE 2.1 – Architecture globale du système Customer 360

2.2 Technologies Utilisées

primary !15 Technologie	Version	Rôle dans le projet
Neo4j	Community Edition	Base de données graphe — modélisation des relations
PostgreSQL	18	Entrepôt de données relationnel (schéma en étoile)
dbt	1.12.0	Transformation et orchestration des modèles SQL
Python	3.13	Algorithme Louvain (NetworkX), scripts d'import
Power BI	Desktop	Dashboard interactif et visualisation des KPIs
Docker	Desktop	Conteneurisation de Neo4j

TABLE 2.1 – Stack technologique du projet

2.3 Pipeline de Données

Le pipeline complet suit les étapes suivantes :

1. **Ingestion** : Import du fichier CSV `Comprehensive_Banking_Database.csv` (5 000 enregistrements) dans la table brute `raw.raw_transactions` de PostgreSQL
2. **Staging** : Nettoyage et typage des données via le modèle dbt `stg_banking_data`
3. **Modélisation** : Création du schéma en étoile (4 dimensions + 1 table de faits) via dbt
4. **Graphe** : Import dans Neo4j et création des nœuds/reliations via Cypher
5. **Communautés** : Détection par l'algorithme Louvain via Python/NetworkX
6. **Liaison** : Vue matérialisée connectant graphe et entrepôt
7. **Visualisation** : Dashboard Power BI connecté à PostgreSQL

3. Modélisation du Graphe Neo4j

3.1 Présentation de Neo4j

Neo4j est une base de données orientée graphe qui stocke les données sous forme de nœuds et de relations. Contrairement aux bases relationnelles, elle excelle dans la représentation de connexions complexes entre entités. Dans notre projet, elle permet de modéliser naturellement le réseau client bancaire.

3.2 Modèle de Graphe

3.2.1 Nœuds (Nodes)

Le graphe contient les types de nœuds suivants :

primary Label	Propriétés	Description
Customer	id, firstName, lastName, age, gender, city, email	Client bancaire
Branch	id, name	Agence bancaire
Transaction	id, amount, date, isAnomaly	Transaction financière
LoanType	name	Type de prêt (Personal, Mortgage, Auto)
CardType	name	Type de carte (Gold, Platinum, Standard)

TABLE 3.1 – Nœuds du graphe Neo4j

3.2.2 Relations

primary !15 Relation	Source	Cible	Propriétés
MADE_WITHDRAWAL	Customer	Transaction	—
MADE_DEPOSIT	Customer	Transaction	—
MADE_TRANSFER	Customer	Transaction	—
OCCURRED_AT	Transaction	Branch	—
BORROWED	Customer	LoanType	loan_id, amount, sta- tus
HOLDS_CARD	Customer	CardType	card_id, limit

TABLE 3.2 – Relations du graphe Neo4j

3.3 Import des Données dans Neo4j

L'import a été réalisé via des requêtes Cypher utilisant `LOAD CSV`. Voici les requêtes principales :

3.3.1 Création des nœuds Client

```

1 LOAD CSV WITH HEADERS FROM 'file:///data-2.csv' AS row
2 CREATE (:Customer {
3     id: toInteger(row.customer_id),
4     firstName: row.first_name,
5     lastName: row.last_name,
6     age: toInteger(row.age),
7     gender: row.gender,
8     city: row.city,
9     email: row.email_address
10 });
11 -- R sultat : Created 5 000 nodes, set 30 000 properties

```

3.3.2 Création des Relations Transactions

```

1 LOAD CSV WITH HEADERS FROM 'file:///data-2.csv' AS row
2 MATCH (c:Customer {id: toInteger(row.customer_id)})
3 MATCH (b:Branch {id: toInteger(row.branch_id)})
4 CREATE (t:Transaction {
5     id: toInteger(row.transaction_id),
6     amount: toFloat(row.transaction_amount),
7     date: row.transaction_date,
8     isAnomaly: toInteger(row.anomaly)
9 })
10 CREATE (t)-[:OCCURRED_AT]->(b)

```

```

11 FOREACH (_ IN CASE WHEN row.transaction_type = 'Withdrawal'
12     THEN [1] ELSE [] END |
13     CREATE (c)-[:MADE_WITHDRAWAL]->(t));
14 -- R sultat : Created 5 000 nodes , 10 000 relationships

```

3.4 Visualisation du Graphe

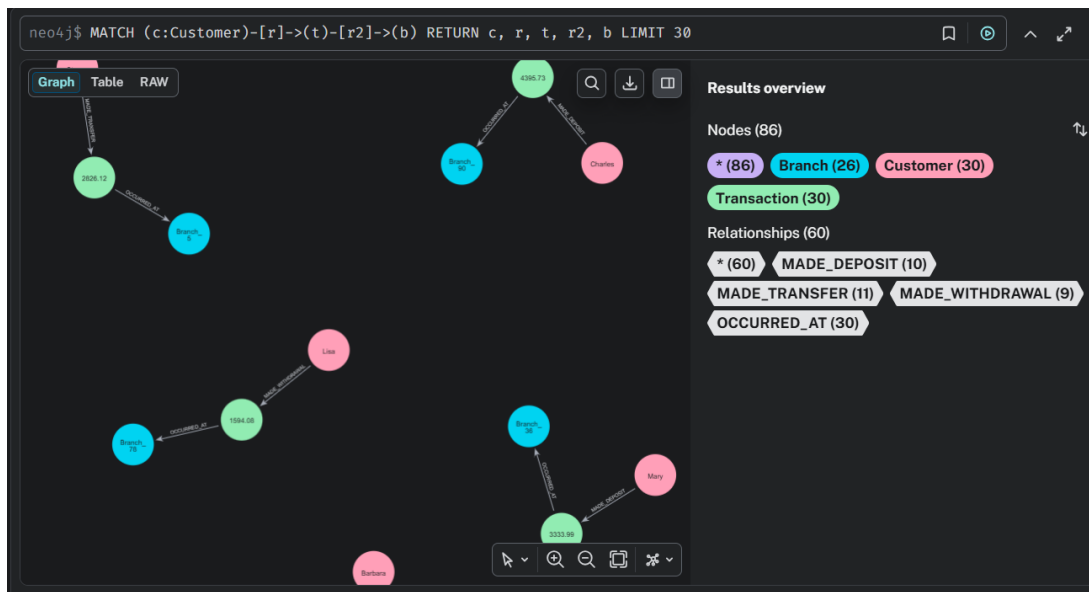


FIGURE 3.1 – Visualisation du graphe Customer 360 dans Neo4j Browser

```
neo4j$ MATCH (n) RETURN labels(n), count(n)
```

	labels(n)	count(n)
1	["Customer"]	5000
2	["Branch"]	99
3	["Transaction"]	5000
4	["LoanType"]	3
5	["CardType"]	3

FIGURE 3.2 – Statistiques des nœuds et relations dans Neo4j

4. Entrepôt de Données PostgreSQL

4.1 Présentation de l'Architecture en Étoile

L'entrepôt de données adopte un **schéma en étoile** (Star Schema), modèle standard en Business Intelligence. Ce schéma place la table de faits au centre, entourée des tables de dimensions qui lui sont directement reliées.

4.2 Schéma en Étoile — Customer 360

4.3 Tables de l'Entrepôt

4.3.1 Table de Faits : fct_transactions

primary!15 Colonne	Type	Description
transaction_id	INTEGER	Clé primaire
customer_id	INTEGER	Clé étrangère → dim_customers
branch_id	INTEGER	Clé étrangère → dim_branches
date_id	DATE	Clé étrangère → dim_time
transaction_type	TEXT	Clé étrangère → dim_transaction_type
transaction_amount	DECIMAL(12,2)	Montant de la transaction
anomaly	INTEGER	Indicateur d'anomalie (0/1)

TABLE 4.1 – Structure de la table de faits fct_transactions

4.3.2 Dimensions

primary !15 Table	Cardinalité	Colonnes principales
dim_customers	5 000 lignes	customer_id, nom, âge, ville, solde, type de carte, prêt
dim_branches	99 lignes	branch_id, branch_name
dim_time	365 lignes	date_id, année, mois, trimestre, nom du jour
dim_transaction_type	3 lignes	type_id (Withdrawal, Deposit, Transfer)

TABLE 4.2 – Tables de dimensions de l'entrepôt

4.4 Vue Matérialisée — Lien Graphe et Entrepôt

La vue matérialisée `raw.materialized_view` constitue le pont entre le graphe Neo4j (communautés Louvain) et l'entrepôt PostgreSQL.

```

1  -- Mod le dbt : materialized_view.sql
2  WITH raw_data_1 AS (
3      SELECT * FROM raw.stg_communities -- R sultats Louvain
4  )
5  SELECT
6      c.customer_id,
7      com.community_id
8  FROM dim_customers AS c
9  LEFT JOIN raw_data_1 AS com
10     ON c.customer_id = com.customer_id

```

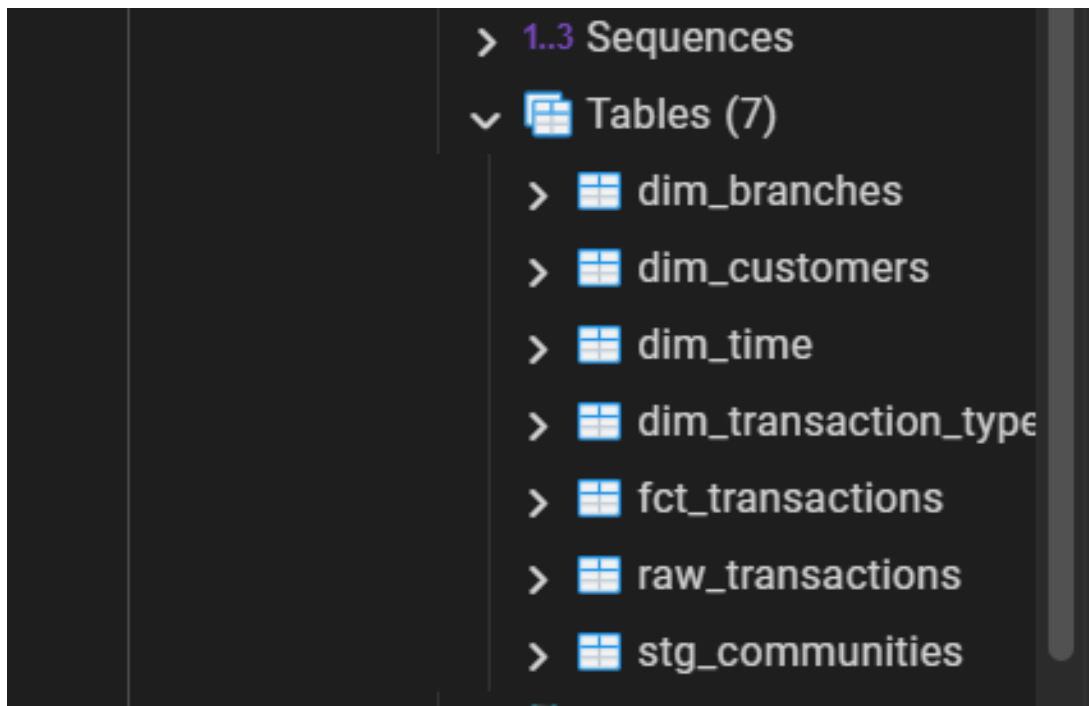


FIGURE 4.1 – Tables du schéma raw dans pgAdmin

5. Transformations dbt

5.1 Présentation de dbt

dbt (Data Build Tool) est un outil de transformation de données qui permet d'écrire des transformations SQL sous forme de modèles versionables. Il gère les dépendances entre modèles, documente automatiquement le pipeline et teste la qualité des données.

5.2 Structure du Projet dbt

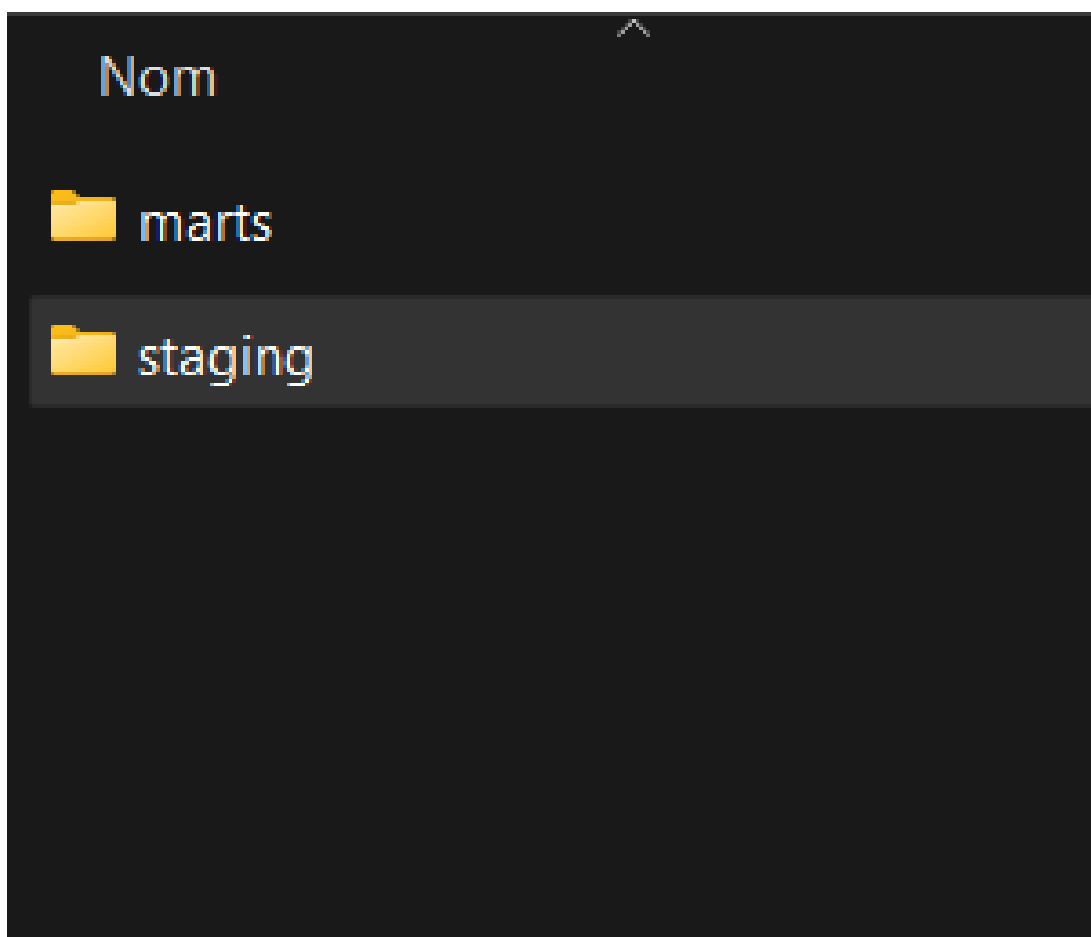


FIGURE 5.1 – Structure du projet dbt

5.3 Couche Staging : stg_banking_data

Le modèle de staging réalise le nettoyage et le typage des données brutes :

```

1  -- models/staging/stg_banking_data.sql
2  WITH raw_data AS (
3      SELECT * FROM raw.raw_transactions
4  )
5  SELECT
6      CAST(customer_id AS INTEGER) AS customer_id,
7      first_name, last_name,
8      CAST(age AS INTEGER) AS age,
9      CAST(account_balance AS DECIMAL(12,2)) AS account_balance,
10     (TRIM(transaction_date))::DATE AS transaction_date,
11     CAST(transaction_amount AS DECIMAL(12,2)) AS
12         transaction_amount,
13     CAST(branch_id AS INTEGER) AS branch_id,
14     CAST(anomaly AS INTEGER) AS anomaly,
15     'Branch_' || CAST(branch_id AS TEXT) AS branch_name
16 FROM raw_data
17 WHERE transaction_id != 'TransactionID'

```

5.4 Exécution du Pipeline dbt

```

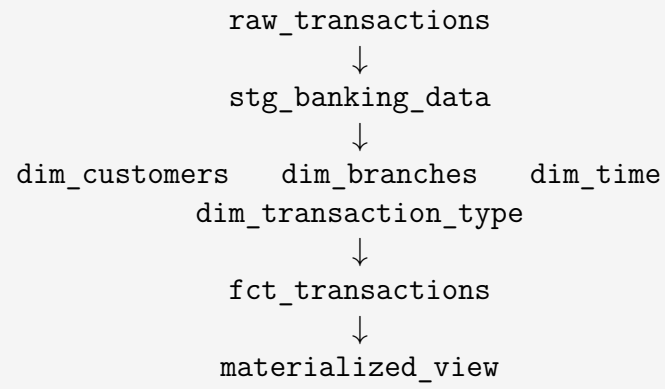
PS C:\Users\MDT\Documents\ISIBD S8\systeme bussines intelligence\bank_bi\bank_bi\bank_bi> dbt run
23:43:36 Running with dbt=1.12.0-b1
23:43:39 Registered adapter: postgres=1.10.0
23:43:45 Found 7 models, 2 sources, 475 macros
23:43:45 Concurrency: 4 threads (target='dev')
23:43:45
23:43:53 1 of 7 START sql view model raw.stg_banking_data ..... [RUN]
23:43:54 1 of 7 OK created sql view model raw.stg_banking_data ..... [CREATE VIEW in 0.98s]
23:43:54 4 of 7 START sql table model raw.dim_time ..... [RUN]
23:43:54 2 of 7 START sql table model raw.dim_branches ..... [RUN]
23:43:54 5 of 7 START sql table model raw.dim_transaction_type ..... [RUN]
23:43:54 3 of 7 START sql table model raw.dim_customers ..... [RUN]
23:43:55 5 of 7 OK created sql table model raw.dim_transaction_type ..... [SELECT 3 in 0.79s]
23:43:55 4 of 7 OK created sql table model raw.dim_time ..... [SELECT 365 in 0.82s]
23:43:55 2 of 7 OK created sql table model raw.dim_branches ..... [SELECT 99 in 0.83s]
23:43:55 6 of 7 START sql table model raw.fct_transactions ..... [RUN]
23:43:55 3 of 7 OK created sql table model raw.dim_customers ..... [SELECT 5000 in 0.91s]
23:43:55 7 of 7 START sql view model raw.materialized_view ..... [RUN]
23:43:55 7 of 7 OK created sql view model raw.materialized_view ..... [CREATE VIEW in 0.27s]
23:43:55 6 of 7 OK created sql table model raw.fct_transactions ..... [SELECT 5000 in 0.52s]
23:43:55
23:43:55 Finished running 5 table models, 2 view models in 0 hours 0 minutes and 10.42 seconds (10.42s).
23:43:56
23:43:56 Completed successfully
23:43:56
23:43:56 Done. PASS=7 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=7
PS C:\Users\MDT\Documents\ISIBD S8\systeme bussines intelligence\bank_bi\bank_bi\bank_bi>

```

FIGURE 5.2 – Résultat de l'exécution dbt run — 7 modèles créés avec succès

5.5 Graphe de Dépendances dbt (DAG)

Les modèles dbt s'enchaînent selon un graphe acyclique dirigé (DAG) :



6. Détection de Communautés — Algorithme Louvain

6.1 Principe de l'Algorithme Louvain

L'algorithme de Louvain est une méthode de détection de communautés dans les graphes, basée sur la maximisation de la **modularité**. Il procède en deux phases itératives :

1. **Phase locale** : chaque nœud est assigné à la communauté de son voisin qui maximise le gain de modularité
2. **Phase d'agrégation** : les communautés sont agrégées en super-nœuds, et le processus est répété

La modularité Q est définie par :

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

où A_{ij} est la matrice d'adjacence, k_i le degré du nœud i , m le nombre total d'arêtes et $\delta(c_i, c_j)$ vaut 1 si i et j sont dans la même communauté.

6.2 Implémentation avec NetworkX et Python

```
1 import networkx as nx
2 from neo4j import GraphDatabase
3 import community.community_louvain as community_louvain
4 import pandas as pd
5
6 # Connexion Neo4j
7 uri = "bolt://localhost:7687"
8 driver = GraphDatabase.driver(uri, auth=("neo4j", "password"))
9
10 # Extraction des relations depuis Neo4j
11 def fetch_relationships(tx):
12     query = """
13     MATCH (c:Customer)-[r1]->(t:Transaction)-[r2:OCCURRED_AT]->(b
14         :Branch)
15     RETURN c.id AS source, b.name AS target
16     UNION
17     MATCH (c:Customer)-[r:BORROWED|HOLDS_CARD]->(type)
18     RETURN c.id AS source, type.name AS target
19     """
20     return [record.data() for record in tx.run(query)]
```

```
20
21 # Construction du graphe NetworkX
22 G = nx.Graph()
23 with driver.session() as session:
24     results = session.execute_read(fetch_relationships)
25     for record in results:
26         G.add_edge(record["source"], record["target"])
27
28 # Application de l'algorithme Louvain
29 partition = community_louvain.best_partition(G)
30
31 # Export vers CSV
32 customer_data = [
33     {"customer_id": node, "community_id": cid}
34     for node, cid in partition.items()
35     if node in customer_ids
36 ]
37 df = pd.DataFrame(customer_data)
38 df.to_csv("customer_communities.csv", index=False)
```

6.3 Résultats de la Détection de Communautés

	A	B	C	D	E
1	customer_id,community_id				
2	3392,18				
3	1,18				
4	4537,18				
5	2388,18				
6	1646,18				
7	108,18				
8	2341,18				
9	430,18				
0	434,18				
1	1418,18				
2	3211,18				
3	1608,18				
4	4305,18				
5	2017,18				
6	508,18				
7	1461,18				
8	3840,18				
9	1663,18				
0	1519,18				
1	1351,18				
2	4001,18				
3	2538,18				
4	1652,18				
5	3545,18				
6	3772,18				

FIGURE 6.1 – Extrait du fichier customer_communities.csv

7. Dashboard Power BI

7.1 Connexion à PostgreSQL

Power BI Desktop a été connecté directement à la base PostgreSQL `bank_bi` via le connecteur natif. Les tables chargées sont :

- `raw.fct_transactions` — table de faits centrale
- `raw.dim_customers` — dimension clients
- `raw.dim_branches` — dimension agences
- `raw.dim_time` — dimension temporelle
- `raw.dim_transaction_type` — dimension types de transaction
- `raw.materialized_view` — communautés Louvain

7.2 Modèle de Données dans Power BI

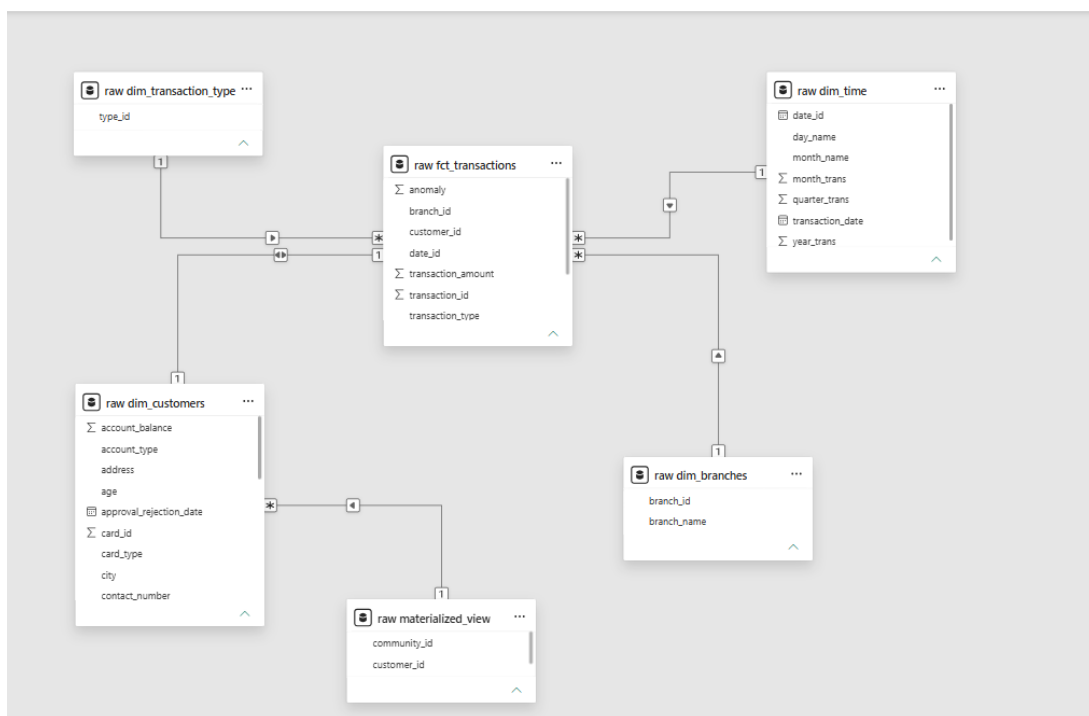


FIGURE 7.1 – Modèle de données Power BI — schéma en étoile

7.3 KPIs Implémentés

Les mesures DAX suivantes ont été créées pour calculer les KPIs métier :

```

1 -- Solde moyen des clients
2 Solde Moyen = AVERAGE(raw_dim_customers[account_balance])
3
4 -- Total des transactions
5 Total Transactions = COUNT(raw_fct_transactions[transaction_id])
6
7 -- Montant total des transactions
8 Montant Total = SUM(raw_fct_transactions[transaction_amount])
9
10 -- Taux d'anomalies
11 Taux Anomalies =
12     DIVIDE(
13         COUNTROWS(FILTER(raw_fct_transactions,
14                             raw_fct_transactions[anomaly] = 1)),
15         COUNT(raw_fct_transactions[transaction_id])
16     )
17
18 -- Turnover par agence
19 Turnover Agence =
20     SUMX(
21         VALUES(raw_dim_branches[branch_name]),
22         [Montant Total]
23     )

```

7.4 Visualisations du Dashboard



FIGURE 7.2 – Dashboard Power BI — Vue principale des KPIs

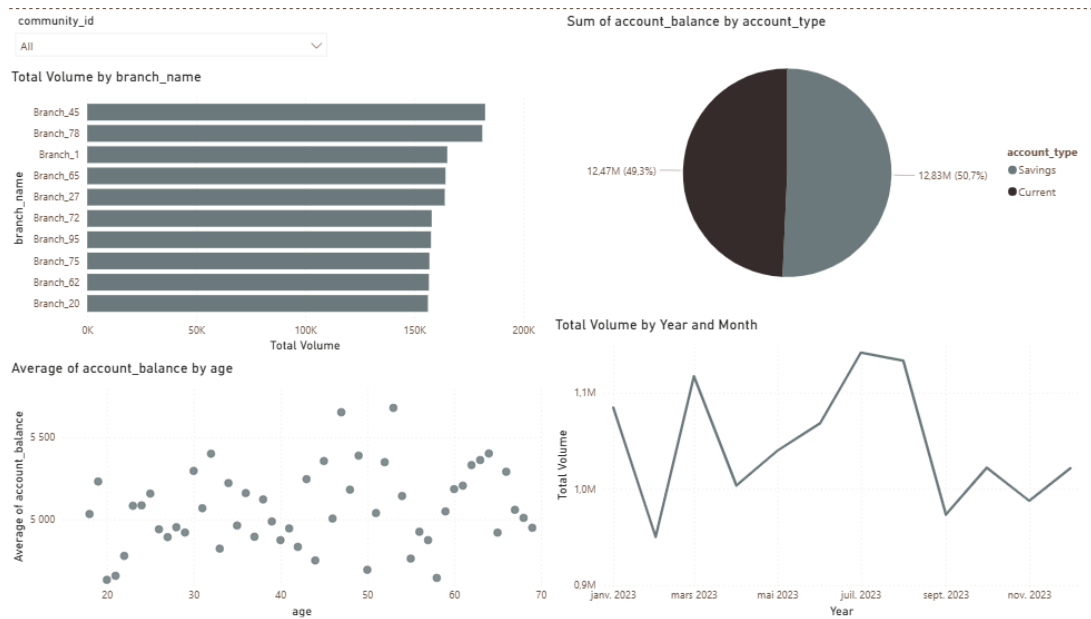


FIGURE 7.3 – Dashboard Power BI — Filtrage par communautés Louvain

8. Tests et Performances

8.1 Tests de Qualité des Données

primary !15 Test	Table	Résultat	Statut
Unicité customer_id	dim_customers	5 000 distincts	OK
Unicité transaction_id	fct_transactions	5 000 distincts	OK
Valeurs NULL anomaly	fct_transactions	0 NULL	OK
Format dates	dim_time	365 dates valides	OK
Intégrité FK branches	fct_transactions	99 agences	OK

TABLE 8.1 – Résultats des tests de qualité des données

8.2 Performances du Pipeline

primary !15 Étape	Durée	Volume
Import CSV → PostgreSQL	< 1 sec	5 000 lignes
dbt run (7 modèles)	2.6 sec	5 000 + dimensions
Import Neo4j (5 requêtes)	≈ 5 min	15 000+ nœuds
Script Louvain Python	< 5 sec	5 000 clients
Chargement Power BI	< 10 sec	7 tables

TABLE 8.2 – Performances mesurées du pipeline

9. Conclusion

9.1 Bilan du Projet

Ce projet a permis de construire une plateforme Customer 360 complète pour une banque, intégrant :

- Un **graphe de relations** dans Neo4j modélisant 5 000 clients, 99 agences, et leurs interactions
- Un **entrepôt de données** en schéma étoile dans PostgreSQL avec 7 modèles dbt
- Une **détection de communautés** par l'algorithme Louvain via Python/NetworkX
- Un **dashboard interactif** Power BI filtrable par communautés